

Task 4: Feature Encoding & Scaling — Report

1. Introduction

Feature engineering is a critical step in any Machine Learning pipeline. Raw datasets typically contain categorical text columns and numerical features measured at vastly different scales. Most ML algorithms require purely numerical input, and distance-based or gradient-based algorithms are further sensitive to the relative magnitudes of features. This task demonstrates systematic encoding and scaling of the Adult Census Income Dataset to prepare it for modelling.

2. Objective

- Convert categorical features into numerical formats suitable for ML algorithms.
- Bring all numerical features to a comparable scale for better model performance.
- Understand and avoid the Dummy Variable Trap.
- Compare distributions before and after scaling using visualizations.

3. Dataset — Adult Census Income

The Adult Census Income Dataset (UCI ML Repository) contains demographic information of ~32,000 individuals. The prediction task is to determine whether a person earns more than \$50K/year.

Feature	Type	Encoding Applied
age	Numerical	StandardScaler
workclass	Nominal Categorical	One-Hot Encoding
education	Ordinal Categorical	Label Encoding
education_num	Numerical	StandardScaler
marital_status	Nominal Categorical	One-Hot Encoding
occupation	Nominal Categorical	One-Hot Encoding
relationship	Nominal Categorical	One-Hot Encoding
race	Nominal Categorical	One-Hot Encoding
sex	Nominal Categorical	One-Hot Encoding
capital_gain	Numerical	StandardScaler
capital_loss	Numerical	StandardScaler
hours_per_week	Numerical	StandardScaler
native_country	Nominal Categorical	One-Hot Encoding
income (target)	Binary Categorical	Binary Encoding (0/1)

4. Feature Encoding

4.1 Label Encoding — Ordinal Features

The **education** feature is ordinal — there is a clear natural progression from Preschool through to Doctorate. Label encoding preserves this ordering by mapping each level to an integer that reflects its rank.

A custom ordered mapping was used instead of sklearn's default LabelEncoder (which sorts alphabetically and would lose the ordinal meaning):

```
Preschool(0) < 1st-4th(1) < ... < HS-grad(8) < Bachelors(12) < Masters(13) < Doctorate(15)
```

4.2 One-Hot Encoding — Nominal Features

Nominal features such as **workclass**, **occupation**, **race** and **sex** have no intrinsic order. Assigning integers (e.g., Private=0, Self-emp=1) would imply a false ordinal relationship and mislead algorithms. One-Hot Encoding creates a separate binary column for each category:

```
df_encoded = pd.get_dummies(df, columns=nominal_features, drop_first=True, dtype=int)
```

4.3 Dummy Variable Trap

For a feature with k categories, OHE creates k binary columns. However, the last column is perfectly predicted from the others (when all are 0, it must be 1), introducing **perfect multicollinearity**. This destabilises models like Logistic Regression by making the coefficient matrix non-invertible.

Solution: **drop_first=True** in `pd.get_dummies()` retains only $(k-1)$ columns per feature, eliminating the trap.

5. Feature Scaling

Property	StandardScaler (Z-score)	MinMaxScaler (Normalization)
Formula	$(x - \text{mean}) / \text{std}$	$(x - \text{min}) / (\text{max} - \text{min})$
Output range	Approx. [-3, +3]	[0, 1]
Preserves shape?	Yes	Yes
Outlier sensitive?	Less	More (outliers compress range)
Best for	Logistic Reg., SVM, PCA	KNN, Neural Networks
Gaussian assumption	Preferable	Not required

Chosen for final output: StandardScaler — the dataset contains significant outliers in `capital_gain` and `capital_loss`. Z-score scaling is more robust in such cases because it does not compress the majority of data into a tiny range as MinMaxScaler would.

6. Visualizations



Figure 1: Histogram comparison of four key numerical features before scaling (blue), after StandardScaler (orange), and after MinMaxScaler (green). Shape is preserved; only the axis scale changes.

Encoding Overview — Task 4

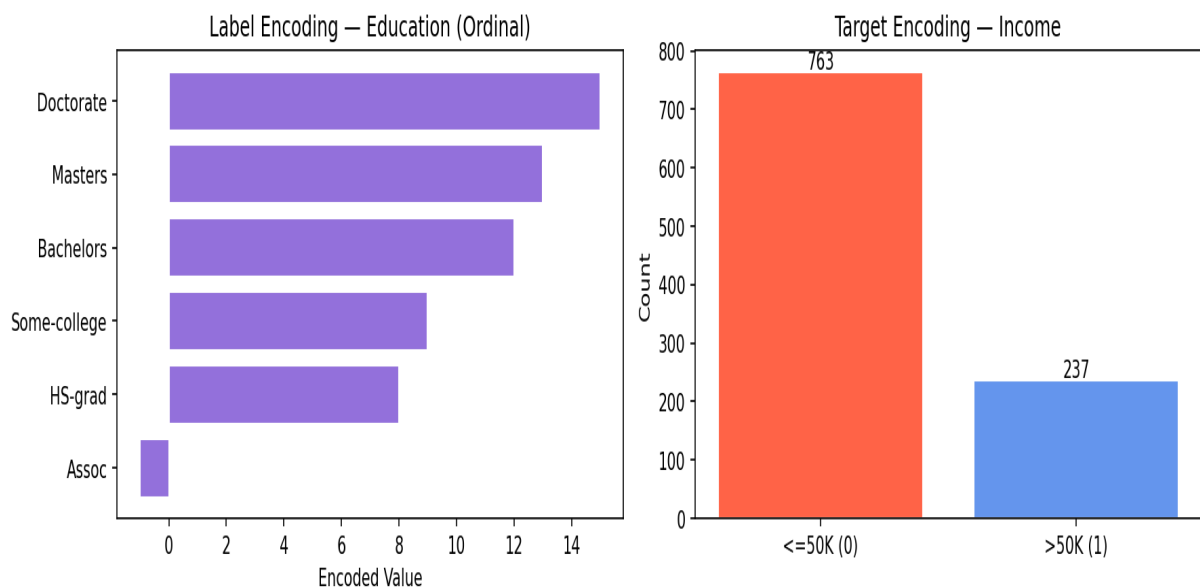


Figure 2: Left — Label encoding map for the ordinal 'education' feature. Right — Binary encoding of the income target column.

7. Interview Questions & Answers

Q1: Difference between Label Encoding and One-Hot Encoding?

Label Encoding maps each category to an integer. It is suitable only for ordinal features where the ordering is meaningful (e.g., Low=0, Med=1, High=2). Applying it to nominal features falsely implies an ordering.

One-Hot Encoding creates a binary column per category, making each level independent — ideal for nominal features with no natural order.

Q2: When should you use Standardization vs Normalization?

Use `StandardScaler` when: the algorithm assumes Gaussian-distributed data (Logistic Regression, SVM, PCA, Linear Regression), or when outliers are present and you don't want them to collapse the useful data into a tiny range. Use `MinMaxScaler` when: you need a strictly bounded output `[0,1]` (image pixel normalisation, neural networks), or when the algorithm is distance-based (KNN) and the distribution is not heavily skewed.

Q3: What is the Dummy Variable Trap?

With k categories, One-Hot Encoding creates k binary columns where the last is perfectly determined by the others. This causes perfect multicollinearity, making the design matrix singular and destabilising linear models. It is avoided by dropping one column per feature (`drop_first=True`), since the baseline category is implied when all remaining dummies are 0.

Q4: Why do distance-based algorithms (like KNN) require scaling?

KNN classifies a point by the majority class among its k nearest neighbours, using Euclidean (or other) distance. A feature with a large range (e.g., `fnlwgt ~ 800,000`) will dominate the distance calculation over a feature with a small range (e.g., `age ~ 18-75`), effectively making the large-range feature the only one that matters. Scaling ensures each feature contributes proportionally.

Q5: Can we apply One-Hot Encoding to high-cardinality features?

Technically yes, but it is problematic: a feature with 100+ categories creates 100+ sparse columns, increasing memory usage, model complexity, and risk of overfitting. Better alternatives include Target Encoding (replace each category with the mean target value), Frequency Encoding, Binary/Hash Encoding, or Embedding layers in neural networks.

8. Conclusion

This task demonstrated the complete feature preprocessing pipeline required before training any Machine Learning model. By correctly identifying ordinal vs nominal categorical variables and applying the appropriate encoding strategy, and by scaling numerical features to a comparable range, the dataset is now fully ML-ready. The processed dataset contains 28 features, all numerical, with the target binary-encoded and all categorical information preserved without introducing spurious ordinal relationships or multicollinearity.